LightGBM with ReLU Loss Function Trained on NYS Standardized Testing Outcomes: After Action Report

Jeremy Gomez

In this after-action report I will briefly describe each step of my pipeline architecture and the steps I took to re-use the provided data while avoiding model overfitting. I will then discuss my investigation of systemic errors made by the model, insights gleaned from my model about the importance of various features, and a discussion of ways I would improve my model based on my investigation of how the model predictions compare to the unblinded test data.

Review of Pipeline Architecture and Model Decisions

Data Reuse:

To use my labeled data efficiently while guarding against overfitting, I created an 80-20 train-test split. I then used Optuna (an automatic hyperparameter tuning library) to tune my LightGBM hyperparameters with 6-fold cross validation inside each trial, each trial was evaluated as the average MSE across six folds allowing Optuna to make more reliable hyperparameter selections. Finally, I retrained the model on the full (train + test) dataset. Because the hyperparameters were selected using cross-validated performance on held-out folds, I had confidence they would generalize, so I retrained the final model on the full dataset to give it 25% more training data than the tuning runs had access to. This final change resulted in a large MSE reduction reducing the MSE of my prediction by 10 points when tested on the unblinded competition test set.

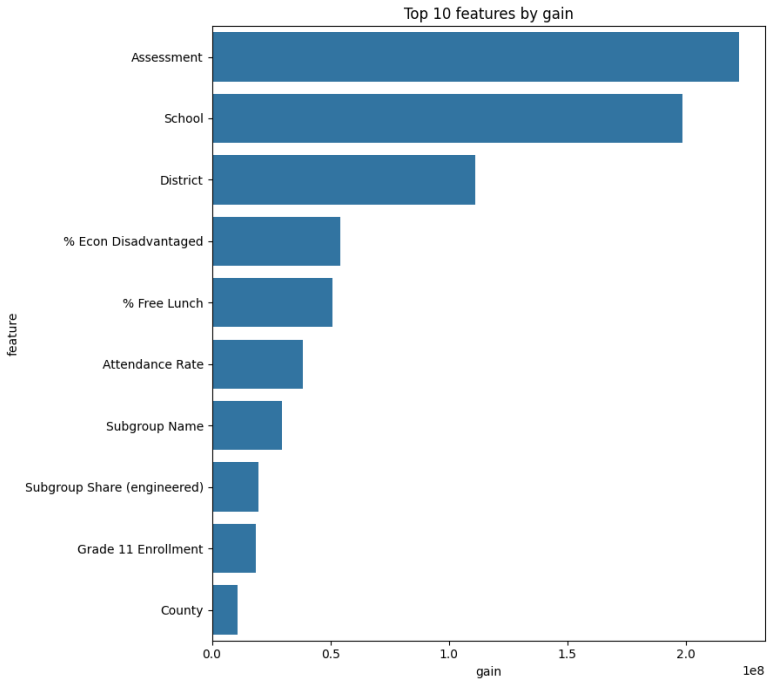
Data Cleaning and Imputation:

I began by loading the competition data and conducting a left join of the training data to the provided 'Schools' and 'District' datasets to add all the available features to the training set while keeping the unmatched schools and districts out of the set. With this expanded set of features, I then analyzed the data for cleaning and imputation. My analysis of missing data informed me that several features had very high missing data rates, five features had missing data in more than 60% of observations. The remaining features had missing data at much lower rates of less than 12%. Luckily all the features with missing data were numeric features, none of the categorical features had any observations with missing data which simplified my imputation strategy.

I decided that dropping the five features identified as having high rates of missing data was prudent as I anticipated these features would have low reliability and may influence the model in unintended ways (i.e. the model fitting a pattern between intact and N/A values to our final prediction). I imputed the remaining numerical features by substituting missing values with median values for observations from the same school. If all observations from the same school had missing values, I would then substitute the missing values with the statewide median for all observations in the dataset. Once these imputation steps were complete, I was able to begin engineering features to increase my model's predictive accuracy.

Feature Engineering:

I created five features that contributed less than I expected to my model. Four of these features were means or ratios of provided features and one feature was the sum of federal and local funding per pupil.



Based on a feature importance by gain chart of my final LightGBM model, only one of the features I created was one of my ten most important features by gain. The feature 'Subgroup Share' (the fraction of students in the school taking the exam) was the ninth most important feature by gain and its contribution to the model was much lower than the three high cardinality categorical variables. I created the 5 features, 'Total Funding per Pupil', 'Federal Funding Ratio', 'Mean Class Size', 'Student Teacher Ratio', and 'Subgroup Share' to reduce the number of splits the LightGBM model needed to make to come to an accurate prediction but I failed to use the creation of these features as an opportunity to cull their redundant component features. I will discuss this in detail in the reflections portion of my report.

LightGBM using a ReLU Loss:

I decided that for the provided dataset, a decision-tree based model such as XGBoost or LightGBM was optimal due to the high cardinality of the categorical variables, the non-linear effects of our features on the subgroup proficiency rate and the large interaction affects between features. I decided to use LightGBM because of its increased training speed over XGBoost.

For regression tasks such as predicting proficiency rates, LightGBM uses an L2 (MSE) loss function by default. Because the L2 loss function applies no penalty binding minimum and maximum proficiency rates I substituted a scaled ReLU loss function that introduced a significant penalty for predictions of less than a 0% and greater than 100%. In testing I found

that this provided a significant MSE improvement compared to clipping the predictions made by the LightGBM using the standard L2 loss function that fell beyond our proficiency rate constraints.

Hyperparameter Training with Optuna:

A downside to using LightGBM and other tree-based machine learning techniques is that it is difficult to tune the model's hyperparameters in a way that will provide accurate predictions for the regression task at hand. For the task of tuning my LightGBM model, I used Optuna. Optuna is an open-source automatic hyperparameter tuning library that uses Tree-structured Parzen Estimators, a form of Bayesian Optimization, to adjust the hyperparameters of the LightGBM model after each trial based on the results of the previous trial. I chose to automatically tune the LightGBM hyperparameters because of the time constraints for this project. Using a method such as grid search would have required an exponential number of trials and many hours of run time to complete. In testing I found that Optuna's performance increased as more trials were conducted with performance topping off at about 30 trials which for my model took about 3 hours to run using Google Collab's high ram CPU connection. Additional Optuna trials after about 30 had limited impact on the predictive ability of my model.

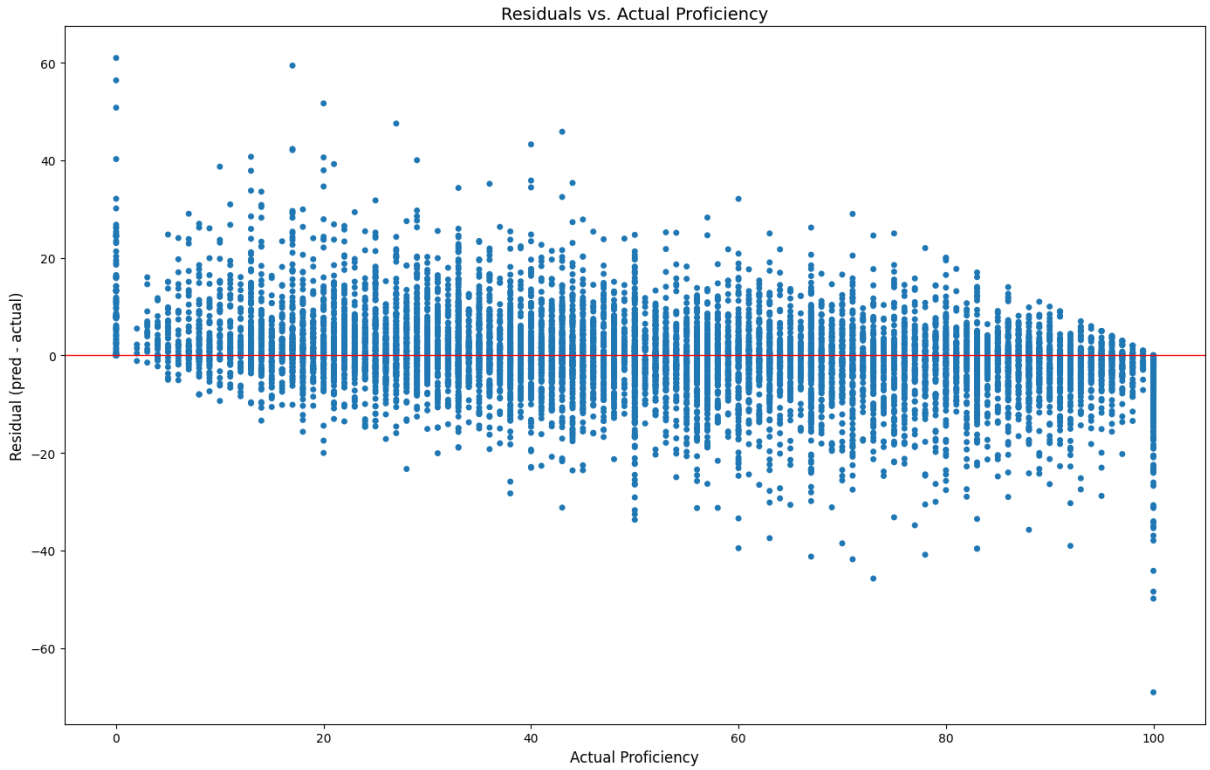
Investigation of Systemic Errors

I began my post competition assessment by investigating areas in which my model made systemic errors when predicting the proficiency scores of New York state student population subgroups. To inform my analysis I created an inner join of the full unblinded data and my lowest MSE Kaggle submission while keeping both the model predicted and actual proficiency rate columns left intact. I then created columns that calculated the residuals and squared errors of each student population and sorted the data frame in descending order by squared error. I did this so that I could quickly view the most and least accurate predictions made by my model to glean any information that could inform my review of systemic errors. At first glance what I noticed was that both the most and least accurate predictions were predicted on student subgroups that that had extreme (0% or 100%) actual proficiency rates. I will discuss this outcome later in my report.

I investigated the overall model bias to verify if my model was systemically over or under predicting proficiency rates by calculating the mean and median residual across predictions made for all student subgroups. The mean residual of my test set is -0.035 while the median is -0.043 thus, I can reasonably conclude that my model does not have a significant positive or negative bias in predicting subgroup proficiency rates.

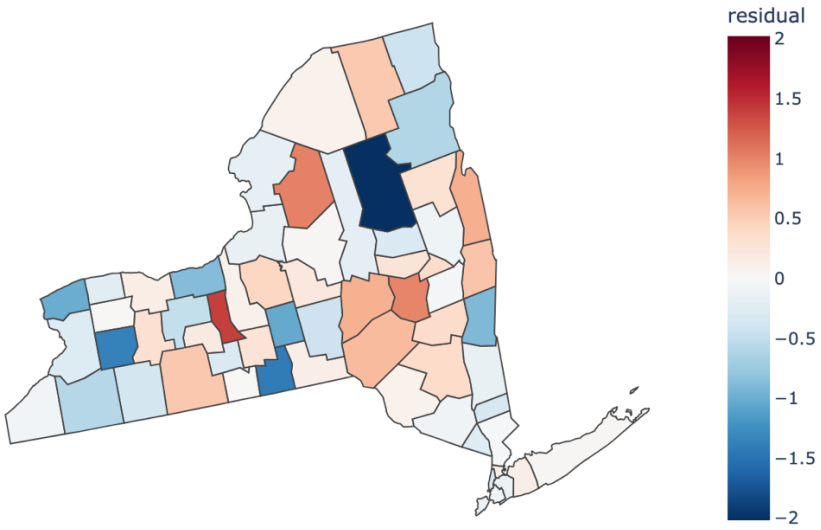
Next, I investigated how my model performs across the range of actual proficiency rates by plotting the actual proficiency by subgroup against the residuals (predicted proficiency – actual proficiency). The plot reveals a clear downward sloping trend, observations with low actual proficiency tend to have large positive residuals, while observations with high actual proficiency tend to have large negative residuals. This pattern is consistent with a bounded target variable since the model tends to regress predications towards the mean. Because proficiency is bounded, predictions for observations near the minimum proficiency rate can

mostly only have positive residuals, and predictions near the ceiling can mostly only have negative residuals.



The spread of residuals is also wider at high and low proficiency values and narrower at higher ones, indicating non-constant error variance (heteroskedasticity) across the range.

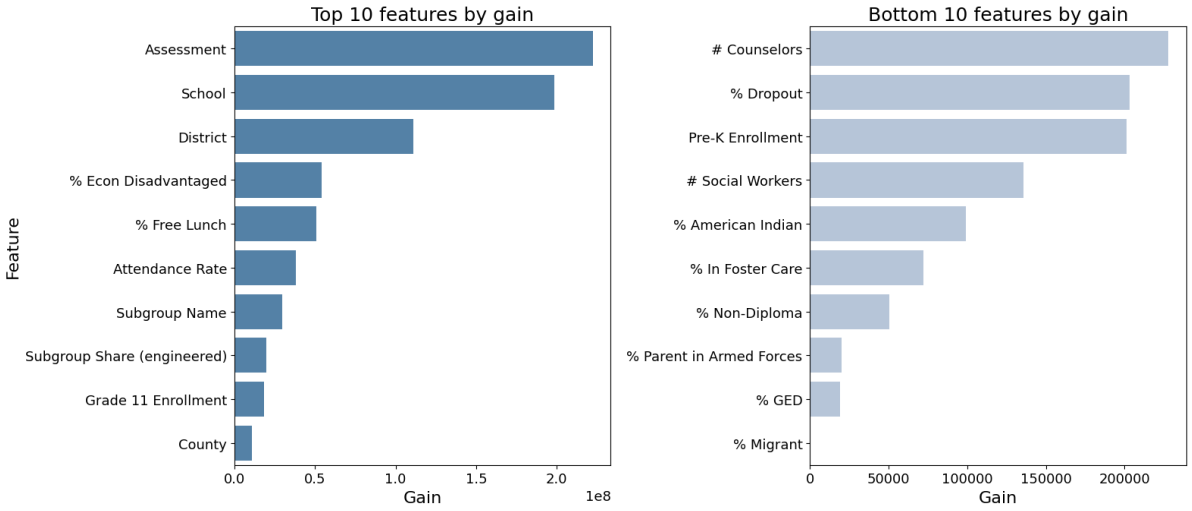
Next I attempted to analyze my model errors by location, to do this I grouped my observations by county and mapped the mean residual of observations in each county to a map of New York State, allowing us to easily assess model error by geographical location.



My takeaway from seeing a map of the mean residual by county is that the model does not have a significant county level bias. The map shows slightly smaller residuals in the downstate regions of New York State, (NYC, Long Island, Hudson Valley) and slightly larger residuals in the remaining regions of the state. However the largest postive and negative mean residuals by county were small (1.400 & -2.018 respectivly) leading me to conclude that the predicitive performance of my model was not significantly impacted by the location of the student subgroups.

Model Insights

To gauge the most important features for predicting proficiency rates across student subgroups I created feature importance by gain charts. Based on the ten most and least important features by gain, the high cardinality categorical variables were the most important for achieving accurate proficiency rates for each subgroup, specifically the 'School', 'Assessment', and 'District' features. This is an intuitive result considering that high-cardinality categorical features are systematically favored by gain-based importance. This is in addition to the fact that no schools or districts were withheld from our training data, meaning that the performance of subgroups from a certain school, district or assessment can be learned from the model and applied to the hidden test data.



The feature importance by gain chart displaying the 10 least important features included features that I did not expect to have small effects on the predicitive ability of our model. I would have anticipated features such as '% in Foster Care', '%Dropout', ' %GED', and '%Migrant' would have differentiated high need (thus low proficiency rate) student subgroups from low need (thus high proficiency rate) subgroups. My hypothesis for why these features were determined to be low importance by the LightGBM model is that the features 'School' and 'District' already introduced the effects of the low importance features in the model. The addition of these features are redundant and offer little additional information because all of the observations from each school already have the school specific features baked in.

Final Reflections and Room for improvement

As I reflect on this assignment after watching the presentations of my classmates and investigating my model's strengths and weaknesses, I have some ideas about how I could have improved my model to increase its predicitive performance. The main ways that I would have adjusted my model to increase its predictive performance while reducing model run times is by focusing more on engineering new features while eliminating redundant features from my training set, and by spending more time investigating ways that I could reduce the systemic errors in my model's residuals.

After reviewing my feature importance by gain charts, I came to the conclusion that I missed an opportunity to reduce my model's training time by engineering new features and not removing their redundant component features from my model. My review of the feature importance by gain chart indicated that many of the features added by the 'School' dataset were already baked into the 'School' feature. Noticing this fact has led me to believe that a deeper investigation of my data at the outset of building my model would have informed me of other opportunities I could have taken to engineer combined features and to drop their constituant features from the model. In future ML projects I will prioritize engineering features by combining component features in ways that will allow me to reduce the number of features my model is trained on. An example of how I could have made this change in my model was by droping the "Federal Funding per Pupil' and 'Local Funding per Pupil' features after engineering the 'Total Funding Per Pupil' feature.

Finally, I want to continue investigating ways to reduce my model's systematic errors. As discussed, the model's errors varied systematically with subgroup performance. My model had large positive residuals for low proficiency rate subgroups and high negative residuals for high performing ones, a pattern consistent with a bounded target and regression toward the mean. In a future ML competition, I would spend more time diagnosing this on the test data from my 80-20 test train split and building a more complex model by pairing a transformed target variable with a training objective to address the residual patterns directly, with a stacking ensemble and meta-learner on top to improve overall predictive performance.

AI Usage Statement

Google Gemini was used as a coding assistant via Google Collab integration while completing this project. No generative AI tools were used for writing or for idea generation.